

HR & Payroll Automation Platform

Project Report — Week 04 Capstone Project

Prepared by:	Humaira
Program:	Python Development Internship — Batch 02
Organization:	Aptura Tech Solutions
Date:	August 2026

1. Objective

The goal of this capstone project is to build a production-style, enterprise-level HR and Payroll Automation Platform demonstrating technical, analytical, documentation, and problem-solving skills. The system implements authentication, role-based access control, employee and attendance management, automated payroll calculation, logging, database backup, and custom error handling — with scalability and security considered throughout.

Project Title:

Enterprise HR & Payroll Automation Platform

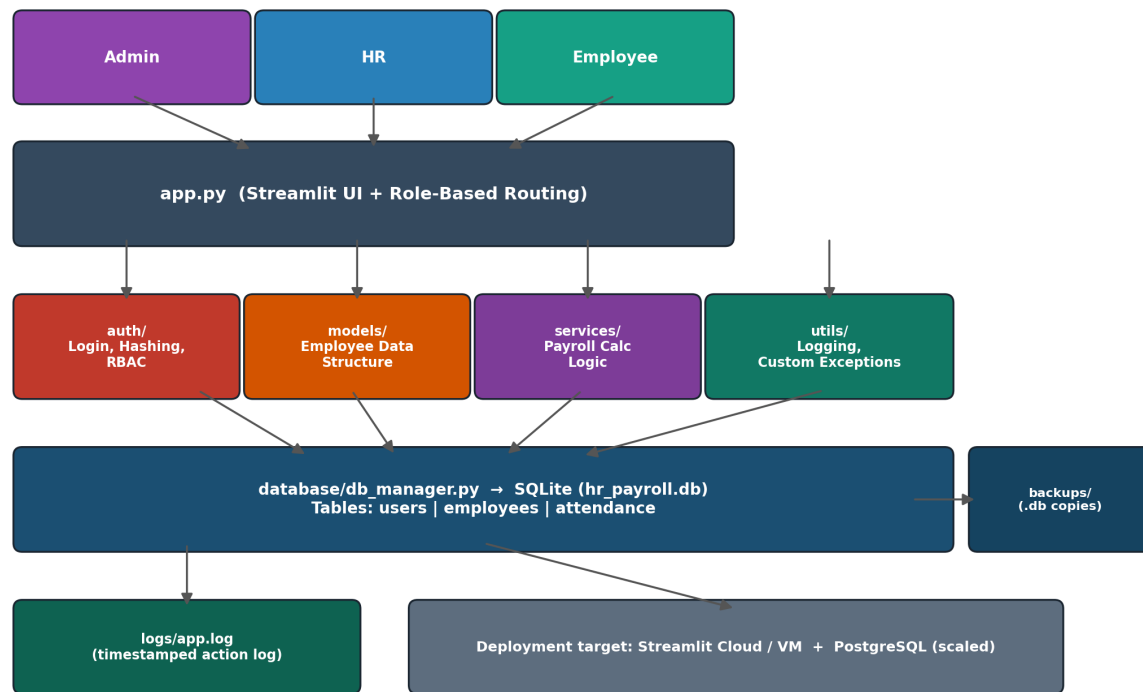
2. Technology Stack

- Language: Python 3.13
- Framework: Streamlit (web-based UI)
- Database: SQLite
- Security: SHA-256 password hashing
- Version Control: Git & GitHub

3. Architecture & Modular Design

The project follows a modular folder structure so that each responsibility — authentication, data models, database access, business logic, and utilities — is isolated into its own module. This mirrors professional, production-style codebases and makes the system easier to test, maintain, and scale.

HR & Payroll Automation Platform — Architecture / Flow Diagram



Flow: User logs in -> app.py checks role via auth/ -> renders allowed modules -> modules read/write via database/db_manager.py -> actions logged to logs/app.log -> Admin can trigger backups/

Flow: A user logs in through app.py, which verifies credentials and role via the auth/ module. Based on role (Admin / HR / Employee), app.py renders only the permitted sections. Each section reads/writes data through database/db_manager.py, which is the single point of contact with the SQLite database. Every significant action is logged to logs/app.log, and Admin users can trigger on-demand database backups stored in backups/.

4. Key Features

- **Authentication & RBAC** — Database-backed login with SHA-256 hashed passwords; three roles (Admin, HR, Employee) each see only the sections relevant to them.
- **Employee Management** — Admin/HR can add employees; adding an employee automatically provisions a login account for them (Employee role).
- **Attendance Management** — Admin/HR mark daily attendance (Present/Absent/Leave); Employees can view only their own attendance history.
- **Payroll** — Admin-only module that calculates salary from basic salary and present days (per-day rate × present days).
- **Database Backup** — Admin-only, one-click timestamped backup of the SQLite database.
- **Logging** — All key actions (employee added, attendance marked, payroll run, backup created) are logged with timestamp and acting user.
- **Custom Exceptions** — Domain-specific error classes (e.g. `InvalidSalaryError`) for clearer error handling than generic exceptions.

5. Expert-Level Questions

Q1. Redesigning the architecture to support 1,000,000 users/records (scalability, availability, security, maintainability)

Scalability: The current SQLite database is a single file and works well for small-scale or demo use, but is not designed for high-concurrency writes at enterprise scale. At 1,000,000+ records, I would migrate to PostgreSQL (or MySQL), which supports proper concurrent connections, indexing, and query optimization. I would add indexes on frequently queried columns (`emp_id`, `username`, `date`) to keep lookups fast as data grows. I would also split the single Streamlit process into a proper client-server model: a backend API (e.g. FastAPI) handling business logic and database access, with the Streamlit/web frontend calling it — this allows the backend to be scaled horizontally (multiple instances behind a load balancer) independently of the UI layer.

Availability: I would deploy on cloud infrastructure (e.g. AWS/Azure) with multiple application instances behind a load balancer, so that if one instance fails, traffic automatically routes to healthy instances. The database would use a managed service with automated failover and read replicas, so read-heavy operations (like viewing employee/attendance lists) don't bottleneck the primary write database.

Security: At this scale, password hashing would move from plain SHA-256 to a slower, salted algorithm designed for passwords (e.g. `bcrypt` or `Argon2`) to resist brute-force attacks. I would introduce session tokens (e.g. JWT) instead of storing plain session state, enforce HTTPS everywhere, add rate-limiting on login attempts, and move all secrets/config out of code into environment variables or a secrets manager.

Maintainability: The existing modular folder structure (`auth/`, `database/`, `models/`, `utils/`) already supports this goal, but at scale I would add a proper test suite (unit tests per module using `pytest`), CI/CD via GitHub Actions to run tests automatically on every push, and API documentation so future developers can extend the system without needing to read all the source code.

Q2. Biggest technical risks and how they were addressed

Risk	Why it matters	How it was addressed / mitigated
SQLite concurrency limits	SQLite locks the whole database file on writes; concurrent writes to the same table (e.g. multiple HR staff making	Acceptable for this stage of the product development
Plain SHA-256 password hashes	SHA-256 is fast, which makes it more vulnerable to brute force / dictionary attacks compared to password hashing algorithms like bcrypt	Used for this stage of development
Role-check logic scattered across the code	Access control (if role == ...) is implemented directly in the code, making it hard to audit and maintain	Mitigated by testing taken in one visible section explicitly, separating
No automatic backup schedule	Backups currently require an Admin to manually make a backup	Make a backup if forgotten, ensure data tested before deployment
Single point of failure (one DB file, one app instance)	If the app instance or process running the app fails, the whole system becomes unavailable	One of the goals for the next development; addressed conceptually

6. Deployment / Implementation Plan

Currently the application runs locally via Streamlit with a local SQLite database. The plan below describes how this would move to a production deployment:

- **Hosting:** Deploy on Streamlit Community Cloud (fastest path, direct from GitHub) or a cloud VM (AWS EC2 / Azure) for more control.
- **Database upgrade:** Migrate from SQLite to PostgreSQL/MySQL to support concurrent users and the scale discussed in the Expert-Level Questions.
- **Secrets management:** Move credentials/config out of source code into environment variables or a cloud secrets manager.
- **HTTPS/Security:** Serve over HTTPS (Streamlit Cloud provides this by default; a VM would use an Nginx reverse proxy with a TLS certificate).
- **Automated backups:** Replace the manual backup button with a scheduled job (cron) writing to cloud storage (e.g. AWS S3) on a daily basis.
- **Monitoring:** Forward the existing logs/app.log output to a centralized logging service (e.g. AWS CloudWatch) for real-time monitoring and alerting.

7. Testing Summary

A full testing report with 20 test cases covering Authentication, RBAC, Employee Management, Attendance, Payroll, Database Backup, and Logging is included as a separate document (tests/test_cases.md in the project repository). All 20 test cases passed. Known limitations identified during testing include: simplified salary calculation (fixed 30-day month, no leave/overtime/tax handling), no password strength validation, no in-app password reset flow, and SQLite's concurrency limits at scale.

8. Deliverables Checklist

- Complete Source Code (GitHub repository)
- Project Report (this document)
- Testing Report (tests/test_cases.md)
- Architecture/Flow Diagram (included above)
- Demonstration Screenshots
- Presentation
- Demo Video (5–10 minutes)

— End of Report —